

Math261 数学分析讲义

第 01 讲 $\sqrt{2}$ 与迭代 (Computing $\sqrt{2}$ by hand)

这门课从一个可以在纸上动手的问题开始：不用计算器上的开方键，怎样把 $\sqrt{2}$ 算到很多位？我们熟悉正平方根 (positive square root) 的记号，但熟悉一个记号，与知道怎样计算、怎样检查计算，还是不同的事。本讲从分数计算开始，逐步检查近似值的精度。

AI 可以协助长除法、列表和画图。使用这些结果时，先估计数量级，再用能独立检查的等式或不等式核对：近似值的平方离 2 有多远？图上的交点是否满足方程？本讲将给出两种具体的检查方法。

本讲研究由同一递推规则生成的数列 (sequence)，先观察各项的小数变化。本讲会完整证明一个否定性的事实：没有有理数 (rational number) 的平方等于 2。至于这些分数似乎共同指向的对象，先用熟悉的 $\sqrt{2}$ 记号称呼它；实数的构造留到第 14 讲；本讲先研究这些有理近似值。

本节新词 (Vocabulary)

English	中文
positive square root	正平方根
sequence	数列
rational number	有理数

1 Locate the target first

计算之前先知道答案的大小。这里的十进制小数 (decimal expansion) 不必写得很长：相邻的几个有限小数 (terminating decimals) 已经可以排除荒唐的结果。若 $0 < u < v$ ，则 $v^2 - u^2 = (v-u)(v+u) > 0$ ；所以比较正数的平方，就能判断大小的方向。

先猜一猜 (Guess first) — 先写预测，再动笔

$\sqrt{2}$ 在 1.4 与 1.5 之间的什么地方？更靠近哪一端？如果只许再做两次平方运算，你会试哪两个小数？在纸上画一小段数轴 (number line)，先标出预测的位置。

词语提示

endpoint: 端点; *predict*: 预测; *finite*: 有限的; *exact*: 精确的。

Example 1.1 — A location check with exact decimals

Predict which end of each row lies below the target. Then square the endpoints. Every entry in this table (表) is exact.

Left endpoint	Its square	Right endpoint	Its square
1.4	1.96	1.5	2.25
1.41	1.9881	1.42	2.0164
1.4142	1.99996164	1.4143	2.00024449

The target belongs between the two endpoints in every row. In particular, an answer such as 1.42 is already visibly too large. These are finite arithmetic (算术的) checks: no claim about an endless process is needed.

我们已经知道该到哪里找. 下一步的困难是: 逐位试小数能行, 但有没有一条规则, 能让一次计算带来不止一位的新信息?

本节新词 (Vocabulary)

English	中文
decimal expansion	十进制小数
terminating decimals	有限小数
number line	数轴
endpoint	端点
predict	预测
finite	有限的
exact	精确的
table	表
arithmetic	算术的

2 递推规则 (The iteration rule)

设 (suppose) 当前猜测是正数 x . 我们希望 $x^2 = 2$, 等式两边同除以 x , 也就是希望 x 与 $2/x$ 相等. 两者的乘积是 2: 若 $x^2 < 2$, 则 $(2/x)^2 = 4/x^2 > 2$; 若 $x^2 > 2$, 则 $(2/x)^2 < 2$. 所以两者一个偏小、一个偏大, 平方为 2 的正数若存在, 必夹在它们之间. 受二分法的启发, 取两者的中点, 即算术平均数 (arithmetic mean), 作为新的猜测; 再对新的猜测重复同样的做法, 两个端点会不会越来越接近? 这给出函数 (function)

$$T(x) = \frac{1}{2} \left(x + \frac{2}{x} \right), \quad x > 0.$$

先试算, 不急着保证这个办法一定成功.

先猜一猜 (Guess first) — Accuracy after three updates

从 $a_1 = 1$ 出发, 每次取 $a_{n+1} = T(a_n)$. 先猜 a_4 与 $\sqrt{2}$ 的小数部分会有多少位相同: 一位、两位, 还是更多? 注意: 从 a_1 到 a_4 是三次更新 (updates), 不是四次.

定义 1.2 — 迭代 (Iteration)

给定初值 (initial value) a_1 , 反复使用递推关系 (recurrence relation) $a_{n+1} = T(a_n)$ 产生数列 (sequence), 称为迭代 (iteration). 本讲研究的迭代 (iteration) 是

$$a_1 = 1, \quad a_{n+1} = \frac{1}{2} \left(a_n + \frac{2}{a_n} \right).$$

这里的下标 (index) n 标明项 (term) 的位置.

先把下面的框遮住, 亲手算到 a_4 . 保留分数, 可以把除法的不确定性推迟到最后.

词语提示

numerator: 分子; *summands*: 相加的各项; *updates*: 更新次数.

Example 1.3 — Three updates by hand

Starting with $a_1 = 1$, we obtain

$$\begin{aligned} a_2 &= \frac{1}{2}(1 + 2) = \frac{3}{2}, \\ a_3 &= \frac{1}{2} \left(\frac{3}{2} + \frac{4}{3} \right) = \frac{1}{2} \cdot \frac{17}{6} = \frac{17}{12}, \\ a_4 &= \frac{1}{2} \left(\frac{17}{12} + \frac{24}{17} \right) = \frac{1}{2} \cdot \frac{289 + 288}{204} = \frac{577}{408}. \end{aligned}$$

The two summands in the last numerator differ by only one: $289 - 288 = 1$. Keep this near equality in mind. We will check the square of the resulting fraction and its decimal digits separately.

这里每一步只有加、乘、除, 但除法有没有可能碰到零? 这是先于“算得准不准”的问题.

命题 1.4 — 每一步都有意义 (Well-defined iteration)

上述数列 (sequence) 的每一项都是正有理数 (positive rational number), 因而每一步运算都有意义 (well-defined).

证明

第一项 1 是正有理数. 如果 a_n 是正有理数, 则 $2/a_n$ 有意义且仍是正有理数, 它与 a_n 的平均数也是正有理数. 由数学归纳法 (mathematical induction), 所有项都是正有理数.

这一证明保证了在任意大的有限步中, 我们能一直算下去. 但它还没有保证位数会稳定, 更没有说明最终指向什么. “每一步合法”与“整个过程达到目标”是两个不同的问题.¹

¹数学归纳法对每个 n 证明“第 n 项有意义”; “数列趋于某个数”则是关于整个数列的一句话, 两者是不同类型的陈述. 后者的精确定义在第 2 讲.

本节新词 (Vocabulary)

English	中文
arithmetic mean	算术平均数
function	函数
updates	更新次数
initial value	初值
recurrence relation	递推关系
iteration	迭代
index	下标
term	项
numerator	分子
summands	相加的各项
positive rational number	正有理数
well-defined	有意义
mathematical induction	数学归纳法

3 直接用计算器看绝对误差 $a_n - \sqrt{2}$

先把分数写成小数. 绝对误差 (absolute error) 指近似值与目标之差的绝对值 (absolute value), 这里记为 $e_n = |a_n - \sqrt{2}|$. 这只是一次数值比较, 不是对无穷过程的定义.

词语提示

comparison: 比较; *rounded*: 四舍五入后的.

Example 1.5 — Read the digits before reading the error

The decimal values below are rounded to the displayed places; the fractions are exact. Read the decimals first and predict the size of each error before looking at the last column.

n	Exact a_n	Decimal value	e_n (approximately)
1	1	1.0000000000000000	4.1421×10^{-1}
2	$3/2$	1.5000000000000000	8.5786×10^{-2}
3	$17/12$	1.4166666666666667	2.4531×10^{-3}
4	$577/408$	1.414215686274510	2.1239×10^{-6}
5	$665857/470832$	1.414213562374690	1.5949×10^{-12}

For comparison, the target begins

$$\sqrt{2} = 1.414213562373095 \dots$$

At a_4 , the first five digits after the decimal point agree with the target: 41421. The sixth digit does not. At a_5 , the error is already much smaller than the last error shown for a_4 . The gain is strikingly uneven.

通过使用计算器“作弊”直接计算 e_n , 我们发现, a_4 就已经达到了相当大的准确度. 之所以说是“作弊”, 是因为 $\sqrt{2}$ 不是有理数, 任何由计算器给出的有限数值都只是一个逼近. 因此这个操作

是”逼近的逼近”，谁数字长谁说了算. 为了对这个现象给出一个数学解释，我们的理论分析中只涉及有理数，不直接使用 $\sqrt{2}$ 的数值.

3.1 如何不借助 $\sqrt{2}$ 的小数值，手动估计误差？

自然的想法是：直接考虑 $a_n^2 - 2$.

$a_n^2 - 2$ 只用分数运算就能精确算出. a_2, a_3, a_4 的平方都大于 2. 若 $a_n^2 > 2$ ，则

$$\frac{1}{a_n^2} < \frac{1}{2} \implies \left(\frac{2}{a_n}\right)^2 = \frac{4}{a_n^2} < 2.$$

所以 a_n 与 $2/a_n$ 一个平方大于 2，一个平方小于 2. 若存在平方为 2 的正数（本讲暂用其存在性，第 14 讲补证），正数的平方保持大小关系，它必在 $2/a_n$ 与 a_n 之间. 两端相差

$$a_n - \frac{2}{a_n} = \frac{a_n^2 - 2}{a_n},$$

这就是 a_n 误差的一个上界，只用到 $a_n^2 - 2$.

词语提示

upper bound: 上界; *iterate*: 迭代项; *bracket*: 夹住目标的区间; *width*: 宽度.

Example 1.9 — An exact bracket from the fourth iterate

Guess the size of $a_4^2 - 2$ before expanding the two squares. The calculation is

$$577^2 = 332929, \quad 2 \cdot 408^2 = 332928, \quad a_4^2 - 2 = \frac{1}{166464}.$$

With $a_4 = 577/408$ and $2/a_4 = 816/577$, the bracket is

$$\frac{816}{577} < \sqrt{2} < \frac{577}{408}, \quad \frac{577}{408} - \frac{816}{577} = \frac{1}{235416} \approx 4.2478 \times 10^{-6}.$$

The bracket width is a safe upper bound for the error of its right endpoint. It is not the error itself. Here it is about twice as large as the actual error.

所有端点都是分数，所有不等式都可以逐项复核. 这里没有从一串越来越小的范围中推出“必然存在一个数”的论证；我们只检查了眼前这一次计算.

本节新词 (Vocabulary)

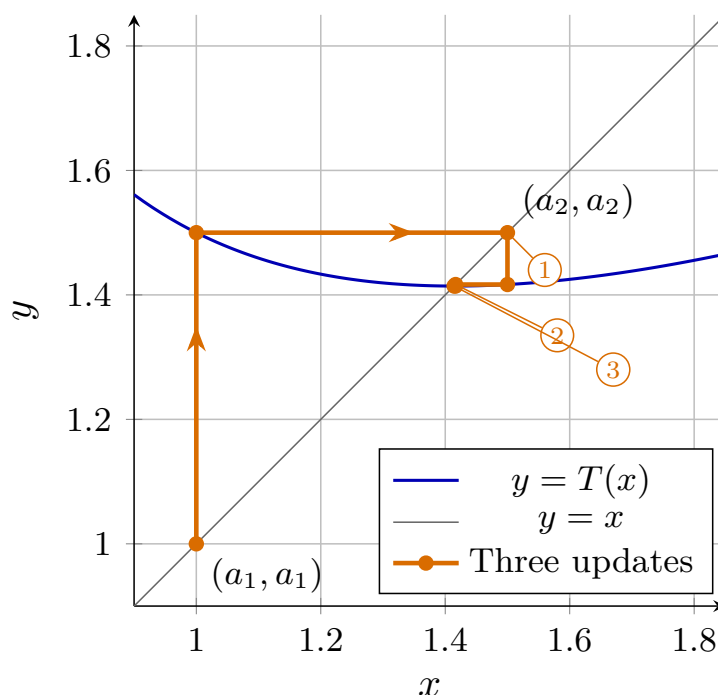
English	中文
absolute error	绝对误差
absolute value	绝对值
comparison	比较
rounded	四舍五入后的
upper bound	上界
iterate	迭代项
bracket	夹住目标的区间
width	宽度

4 把一次更新画出来 (Draw one update)

表格把大小写清楚, 图像 (graph)² 把一次更新如何接到下一次写清楚. 在同一个坐标系 (coordinate system) 内画出 $y = T(x)$ 与直线 $y = x$. 先在对角线 (diagonal) 上取 (a_1, a_1) , 沿竖直方向到函数图像, 再水平走向回对角线.

先猜一猜 (Guess first) — 竖直一步, 水平一步

从 $(1, 1)$ 出发, 第一段应向上还是向下? 第二段应向左还是向右? 到达 (a_2, a_2) 后, 再一次竖直移动会朝哪个方向? 先画箭头, 再看图 1.



先竖直到曲线读取输出, 再水平到对角线把输出变成新输入; 圆圈中的数字依次标出三次更新得到的点 (a_2, a_2) , (a_3, a_3) , (a_4, a_4) , 后两次在交点附近很难分开.

为什么要走回 $y = x$? 水平移动不改变纵坐标, 到对角线时, 横坐标也变成同一个数. 这个动作把“输出”交还给“输入”, 于是可以再次使用同一张函数图像.

词语提示

horizontal: 水平的; *segments*: 线段; *diagonal*: 对角线; *vertical*: 竖直的; *update*: 更新一次.

Example 1.10 — Decode the first two corners

The first vertical segment ends at

$$(1, T(1)) = (1, 3/2).$$

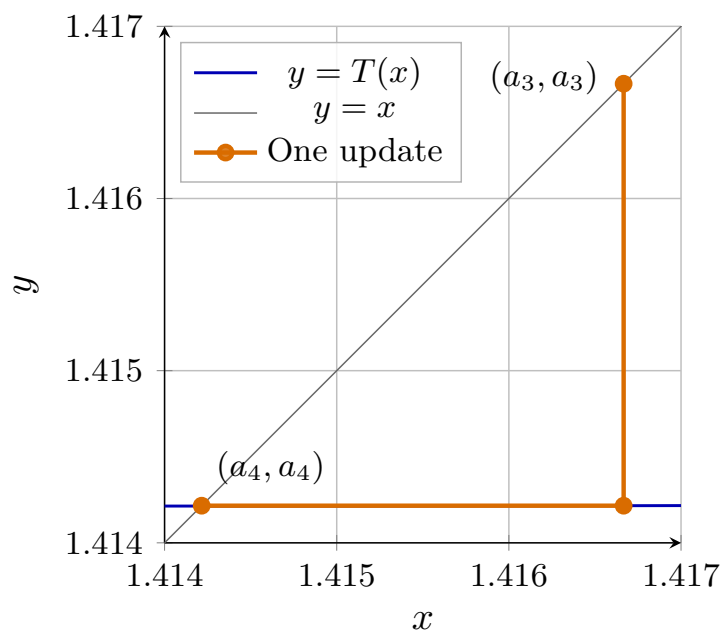
Moving horizontally to $y = x$ gives $(3/2, 3/2)$. The next vertical segment ends at $(3/2, 17/12)$; its endpoint is below the diagonal because $17/12 < 3/2$. The next horizontal segment ends at $(17/12, 17/12)$. Each pair of segments implements one update, not two updates.

²这种作图在第 4 讲称为蛛网图 (cobweb diagram).

图中第一项先向上跳, 此后的几项往回走. 最后几段看不见, 不等于它们长度为零: 纸上的分辨率 (resolution) 已经不够. 带着这个区别再看局部放大图.

4.1 图上的交点意味着什么 (What does an intersection mean?)

图 2 放大第一张图中看不清的部分. 这次不从起点画, 而从 a_3 出发. 横纵坐标使用同一比例, 避免把方向与大小关系画歪.



放大后可看见 a_3 到 a_4 的更新. 在这个窗口里 $y = T(x)$ 已经几乎是一条水平线, 水平那一段几乎压在它上面; a_4 与交点仍难以分辨, 精度需要由算式而非线宽决定.

如果某个输入 c 已经满足 $T(c) = c$, 那么折线就没有可走的距离. 这样的输入称为不动点 (fixed point). 这个词先描述“一次更新不改变什么”, 并不保证从别处出发会走到它.

定义 1.11 — 不动点 (Fixed point)

若输入 c 满足 $T(c) = c$, 则称 c 为函数 (function) T 的不动点 (fixed point). 对于本讲的 T , 输入范围为 $c > 0$.

命题 1.12 — 停止位置的候选 (A candidate stopping value)

正数 c 是 $T(x) = (x + 2/x)/2$ 的不动点 (fixed point) 当且仅当 $c^2 = 2$. 这样的正数至多有一个.

证明

对 $c > 0$, 等式 $c = (c + 2/c)/2$ 两边乘以 $2c$, 得到 $2c^2 = c^2 + 2$, 即 $c^2 = 2$; 这些运算可以逆向进行. 若 $c, d > 0$ 且两者平方均为 2, 则 $(c - d)(c + d) = 0$. 因为 $c + d > 0$, 只能有 $c = d$.

不动点方程给出了唯一可能的正目标. 迭代是否趋近它留待第 4 讲; 眼下可以先回答一个更具体的问题: 某一项能否恰好等于它?

本节新词 (Vocabulary)

English	中文
graph	图像
coordinate system	坐标系
diagonal	对角线
horizontal	水平的
segments	线段
vertical	竖直的
update	更新一次
resolution	分辨率
fixed point	不动点

5 平方根的无理性 (Irrationality of the square root)

每项都是有理数. 如果某一步恰好到达不动点, 就会得到一个平方为 2 的有理数. 这样的分数存在吗? 分母无论多大, 整数的奇偶性都能用于检查这个等式.

先猜一猜 (Guess first) — 大分母能不能消除最后一点误差?

已经算出 $577^2 - 2 \cdot 408^2 = 1$. 你猜能否把两个正整数换得更大, 让这个差恰好变成 0? 如果你认为不能, 理由必须适用于未列出来的整数, 而不只是表中这几个数.

有理数能写成 p/q , 其中 p, q 为整数 (integers)³, $q \neq 0$. 通过约分, 可以让它成为最简分数 (fraction in lowest terms): 分子 (numerator) 与分母 (denominator) 没有大于 1 的公因数 (common factor). 下面只需检查奇偶性 (parity), 即整数为奇数 (odd) 还是偶数 (even), 不需检查分母有多大.

词语提示

identity: 恒等式; *parity*: 奇偶性; *even*: 偶数的; *odd*: 奇数的.

Example 1.13 — Look at parity, not at size

A few squares suggest the right pattern:

$$9^2 = 81, \quad 10^2 = 100, \quad 11^2 = 121, \quad 12^2 = 144.$$

The square of an odd integer is odd; the square of an even integer is even. The crucial step is to replace the sample by an identity. If an integer is odd, write it as $2k + 1$. Then

$$(2k + 1)^2 = 2(2k^2 + 2k) + 1,$$

which is odd. Therefore an integer whose square is even must itself be even. The four numerical samples suggested the rule; the identity proves it for every integer.

注意证明用的方向: “平方为偶数 (even), 所以原数为偶数”. 只说 “偶数的平方是偶数” 还不够, 因为那是另一个方向.

³初中课本把有理数简单说成 “分数”, 这并不准确: 有理数是能写成 p/q 的数, 其中 p, q 为整数, $q \neq 0$. 分子分母不是整数的 “分数” (例如 $\sqrt{2}/2$) 不一定是无理数.

定理 1.14 — $\sqrt{2}$ 的无理性 (Irrationality of $\sqrt{2}$)

不存在有理数 (rational number) r 满足 $r^2 = 2$. 因此, 若用 $\sqrt{2}$ 表示平方为 2 的正数, 则它不是有理数, 而是无理数 (irrational number).

证明

假设存在这样的有理数. 它不为零, 必要时换成它的相反数 (additive inverse), 便可设它为正数. 将它写成最简分数 p/q , 其中 p, q 为正整数. 由 $(p/q)^2 = 2$ 得

$$p^2 = 2q^2.$$

右侧为偶数, 由刚才的奇偶性事实, p 必为偶数. 写 $p = 2k$, 代入得 $4k^2 = 2q^2$, 因而 $q^2 = 2k^2$. 同样, q 也必为偶数. 于是 p 与 q 都能被 2 整除, 与最简分数的选择矛盾. 所以不存在这样的有理数.

因此, 每个有理近似值与目标之间都有非零误差; 此前的区间估计则说明, 这个误差可以很小. 两种结论分别回答了“能否相等”和“相差多少”.

本节新词 (Vocabulary)

English	中文
integers	整数
fraction in lowest terms	最简分数
denominator	分母
common factor	公因数
parity	奇偶性
odd	奇数
even	偶数
identity	恒等式
irrational number	无理数
additive inverse	相反数

6 Use AI to extend the experiment

下面用程序继续计算, 检验前面的速度猜测. 精确分数 (exact fractions) 与小数输出分别保存, 以便复核残差.

词语提示

scientific notation: 科学记数法; *significant digits*: 有效数字; *floating-point*: 浮点的; *denominator*: 分母; *precision*: 计算精度; *magnitude*: 数量级; *at least*: 至少.

AI Lab — Part A: keep the fractions exact

Before running anything, predict the order of magnitude of the errors at a_6 and a_{10} . Then give an AI the following request.

Use exact rational arithmetic to compute $a_1, \dots, a_{10}$ from $a_1 = 1$ and $a_{n+1} = (a_n + 2/a_n)/2$. Return all ten fractions in a plain-text file. Also return a table of $|a_n - \sqrt{2}|$ in scientific notation,

with at least four significant digits. Use high-precision decimal arithmetic, report the precision, and repeat the error calculation at a higher precision. Do not convert a floating-point decimal into a fraction and call it exact.

A reproducible (可复现的) outline in Python is:

```
from fractions import Fraction
from decimal import Decimal, localcontext
a = Fraction(1)
with localcontext() as ctx:
    ctx.prec = 1100
    root = Decimal(2).sqrt()
    for n in range(1, 11):
        x = Decimal(a.numerator) / Decimal(a.denominator)
        error = abs(x - root)
        print(n, a, format(error, ".8E"))
        a = (a + 2 / a) / 2
```

Repeat at precision 1300; the printed errors should agree. At a_{10} , each integer in the fraction has 196 digits: keep them in a separate file.

Questions to answer from the output. Which rows first put the error below 10^{-5} , 10^{-10} , and 10^{-20} ? Does each update add roughly the same number of accurate digits, or does the number roughly double once the error is small? Quote two successive pairs of rows as evidence. Finally, explain why this finite table does not prove a law for all future rows.

科学记数法 (scientific notation) 的指数 (exponent) 让极小误差仍清晰可读. 请先实验, 再用下表核对输出, 检查自己的预测从哪一项开始失准.

n	e_n (approximately)	n	e_n (approximately)
1	4.1421×10^{-1}	6	8.9929×10^{-25}
2	8.5786×10^{-2}	7	2.8593×10^{-49}
3	2.4531×10^{-3}	8	2.8905×10^{-98}
4	2.1239×10^{-6}	9	2.9539×10^{-196}
5	1.5949×10^{-12}	10	3.0849×10^{-392}

表中指数的变化显示了位数的快速增长. 为什么近似翻倍? 第 4 讲回答; 这张有限的表还不能证明它.

6.1 What if the starting value is far away?

一个办法从幸运的初值出发算得快, 还不能说明它面对很差的初值会怎样. 现在把起点向两边推远. 为了让“多少步”有确定含义, 先选一个误差门槛, 再记录首次达到它的项.

先猜一猜 (Guess first) — 远处出发

把起点换成 100, 需要多少次更新, 误差才小于 10^{-5} ? 写一个预测. 再换成 0.01: 它离目标比 100 近得多, 是否就一定更快? 先算两个新起点各自的下一项, 允许自己修改预测, 但保留最初的答案.

词语提示

threshold: 门槛; *indices*: 下标; *index*: 下标.

词语提示

checkpoint: 用于核对的中间结果; *restart*: 从头重新计算.

AI Lab — Part B: compare starts, not just final answers

Repeat Part A for exact starts 100 and 1/100. Save the first ten fractions and errors; continue if the threshold has not been reached. Count updates as $n - 1$.

Your first checkpoints are

$$T(100) = 50.01, \quad T(0.01) = 100.005.$$

The two checkpoints above are exact. In the table below, values are rounded to nine decimal places; check your starting values and indices against them.

n	Start 100: a_n	Start 0.01: a_n
1	100.000000000	0.010000000
2	50.010000000	100.005000000
4	12.552458047	25.026244751
6	3.335281609	6.356201936
8	1.492000890	1.967525444
10	1.414215014	1.416242067

For the threshold $|a_n - \sqrt{2}| < 10^{-5}$, the first successful indices are $n = 4$, $n = 10$, and $n = 11$ for starts 1, 100, and 0.01, respectively. Thus the update counts are 3, 9, and 10.

Questions to answer. Why does “it roughly halves” describe the early start-100 run better than the late part? Why does the small start jump upward? For threshold 10^{-6} , find each first successful index and check the preceding row.

从 100 出发, 前几步主要减小近似值; 接近目标后, 误差才开始快速下降. 从 0.01 出发则先跳到了 100.005. 这解释了为什么仅比较初值与目标的距离, 不足以预测计算需要多少步.

词语提示

residual: 残差; *certify*: 给出严格保证; *stored*: 存储的.

Example 1.16 — A display can stop while exact arithmetic continues

Suppose a program rounds to three decimal places after every update. Near the target it may store 1.414. But exact arithmetic gives

$$T(1.414) \approx 1.414213578501,$$

which rounds back to 1.414. Yet its residual is $(1.414)^2 - 2 = -0.000604 \neq 0$. An unchanged display does not certify a solution: check the residual of the value actually stored.

这就是本实验的核查步骤：比较输出之前，先问程序存的是什么. 在 AI 的说明文字里出现“精确”二字，不能替代精确分数和可复算的残差.

本节新词 (Vocabulary)

English	中文
exact fractions	精确分数
scientific notation	科学记数法
significant digits	有效数字
floating-point	浮点的
precision	计算精度
magnitude	数量级
at least	至少
exponent	指数
threshold	门槛
indices	下标
checkpoint	用于核对的中间结果
restart	从头重新计算
residual	残差
certify	给出严格保证
stored	存储的

7 另一种迭代 (A contrasting iteration)

慢慢需要对照才能看清. 固定作业研究另一个具体数列：

$$b_1 = 1, \quad b_{n+1} = 1 + \frac{1}{b_n}.$$

下面只启动实验，把后面的计算与判断留给你. 别让前一个例子的速度成为未经检查的预期.

先猜一猜 (Guess first) — 会一直待在同一边吗？

先心算 b_2, b_3 . 第三项比第二项小，第四项还会继续变小吗？你猜它会从一侧接近某个位置，还是在两侧交替出现？

词语提示

candidate: 候选值.

Example 1.17 — Start the comparison

The first four terms are

$$b_1 = 1, \quad b_2 = 2, \quad b_3 = \frac{3}{2}, \quad b_4 = \frac{5}{3} \approx 1.666666666667.$$

If a positive input c remains unchanged by this rule, then

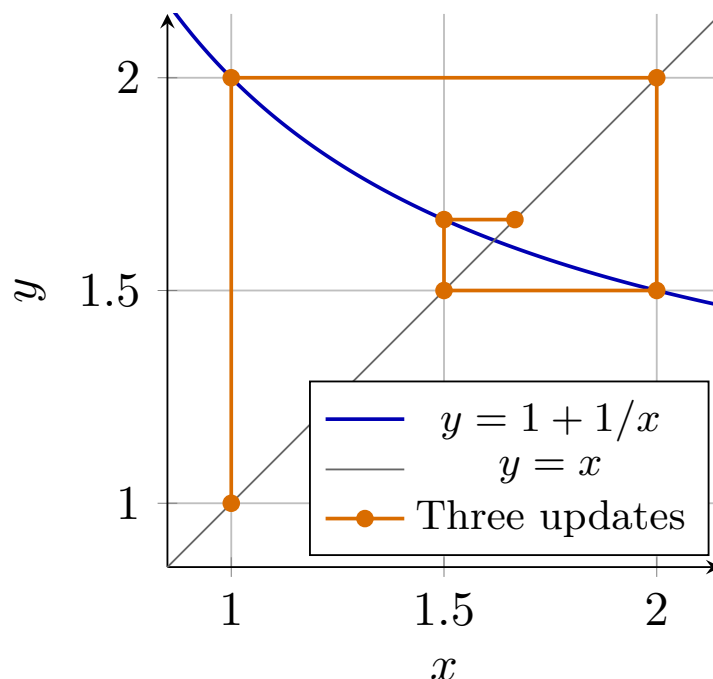
$$c = 1 + \frac{1}{c}, \quad c^2 - c - 1 = 0.$$

Completing the square gives $(2c - 1)^2 = 5$. The positive candidate is

$$\varphi = \frac{1 + \sqrt{5}}{2} \approx 1.618033988750.$$

The first four terms lie below, above, below, and above this candidate. After the same number of updates, $|b_4 - \varphi| \approx 4.8633 \times 10^{-2}$, whereas (相比之下) $|a_4 - \sqrt{2}| \approx 2.1239 \times 10^{-6}$. Predict what another four updates will do before computing them.

这个候选值称为黄金比 (golden ratio). 刚才解方程只找出了候选值; 上下振荡 (oscillation) 是我们在已算出的项里看到的现象. 两者都没有自动回答整个过程将怎样继续. 图 3 把这几项画成折线: 请看拐角落在交点的哪一侧, 以及每一步跨过的距离.



折线的拐角在交点两侧交替出现; 请在作业中继续画到 b_8 , 比较这张图与图 1 的步幅.

同样是“上一项产生下一项”, 这次的折线来回折返, 而且前三次更新后的误差大得多. 下一步要用自己的表检查: 这种对照能否持续? 第 4 讲会把两个实验放在一起解释.

本节新词 (Vocabulary)

English	中文
candidate	候选值
golden ratio	黄金比
oscillation	振荡

8 Exercises

以下习题都从一个预测或一次大小比较开始. 先在纸上留下猜测, 再计算, 再解释. 标有 **home-work** 的题目提交时, 请保留猜错后修改的那一行; 它能说明你看到了什么.

词语提示

residual: 残差; *factoring*: 因式分解; *bounds*: 上下界; *second term*: 第二项; *lowest terms*: 最简形式; *denominator*: 分母; *contradict*: 与.....矛盾.

1.1 ★ The two measurements of error. Guess whether the residual $a_n^2 - 2$ is larger or smaller than the error $a_n - \sqrt{2}$ for $n = 2, 3, 4$. Compute the residuals exactly and explain the comparison by factoring

$$a_n^2 - 2 = (a_n - \sqrt{2})(a_n + \sqrt{2}).$$

Use $1 < \sqrt{2} < 3/2$ to turn each residual into rational upper and lower bounds for the error.

1.2 ★★ (作业) 较慢的交替实验. 设 $b_1 = 1$, $b_{n+1} = 1 + 1/b_n$. 先预测每次更新的方向, 再算出 $b_1, \dots, b_8$ 的精确分数与小数. 把图 3 的折线续画到 b_8 , 由图猜测目标. 然后解方程 $c = 1 + 1/c$, 得到正的候选值 φ , 列出带符号的误差 $b_n - \varphi$. 比较 $|b_8 - \varphi|$ 与 $|a_8 - \sqrt{2}|$. 把表和图带到第 4 讲.

1.3 ★ One update can make two starts agree. Predict which of the starts $2/5$ and 5 has the larger second term under $T(x) = (x + 2/x)/2$. Compute both, then explain the result by comparing $T(x)$ with $T(2/x)$. Find another pair of positive rational starts whose iterates agree from the second term on. Does equality of the later terms imply equality of the starting values?

1.4 ★★ A second route to irrationality. Suppose, only for this problem, that p/q is in lowest terms with positive integers p, q and $(p/q)^2 = 2$. Predict the signs of $2q - p$ and $p - q$ from $1 < p/q < 2$, then verify

$$\frac{2q - p}{p - q} = \frac{p}{q}.$$

Show that $0 < p - q < q$, and explain why a smaller positive denominator contradicts lowest terms.

1.5 ★★★ (作业) 目标可以达到的近邻问题. 把递推改为 $c_{n+1} = (c_n + 4/c_n)/2$, $c_1 = 1$. 先预测正的候选停止值, 再算到 c_4 . 这次候选值是有理数, 这是否迫使迭代恰好命中它? 解正数方程 $(x + 4/x)/2 = 2$, 并据此倒推: 若某一步恰好命中, 前一步必须是多少? 比较“目标是有理数”与“这个迭代能到达目标”这两句话.

1.6 ★★ 另一种逼近 $\sqrt{2}$ 的方法. 对正数 x , 取它的整数部分, 再对余下的部分取倒数, 然后重复这个步骤.

(a) 用计算器对 $\sqrt{2}$ 做这个步骤, 记下依次取出的整数 $m_0, m_1, \dots$, 把前 8 个截断

$$d_1 = m_0, \quad d_2 = m_0 + \frac{1}{m_1}, \quad d_3 = m_0 + \frac{1}{m_1 + \frac{1}{m_2}}, \dots$$

化成最简分数.

(b) 利用 $(\sqrt{2} - 1)(\sqrt{2} + 1) = 1$ 说明每一步的余数相同, 得出把 d_{n+1} 用 d_n 表示的递推公式.

(c) 证明: $|d_n - \sqrt{2}| < 1/q^2$, 其中 $d_n = p/q$. 提示: $d_{n+1} = \frac{p+2q}{p+q}$; 考察 $p^2 - 2q^2$.

(d) 例 1.3 中有 $17^2 - 2 \cdot 12^2 = 1$. 设 $a_n = p/q$ 是 §2 迭代的第 n 项, 写出 a_{n+1} 的分子与分母, 证明 $n \geq 2$ 时 $|a_n - \sqrt{2}| < 1/q^2$. 提示: $a_{n+1} = \frac{p^2 + 2q^2}{2pq}$; 考察 $p^2 - 2q^2$.

1.7 ★★ 立方根. 现在尝试用本节的方法逼近 $2^{1/3}$.

回顾 $\sqrt{2}$ 的逼近过程: 从 $a_0 = 1$ 开始, 找出一个满足 $x^2 > 2$ 的初值 a_1 . 我们的目的是把 a_1 换成更小、更靠近 $\sqrt{2}$ 的 a_2 , 以此递推. 我们知道 a_1 满足 $a_1 > \sqrt{2} > 2/a_1$; 由于 $\sqrt{2}$ 是 a_1 与 $2/a_1$ 的几何平均, 由 AM-GM 不等式, 算术平均 $\frac{1}{2}(a_1 + 2/a_1)$ 可以作为 a_2 . 对于 $2^{1/3}$, 它满足方程 $x^3 = 2$. 类似地, 从 $b_0 = 1$ 开始.

(a) 如何用 AM-GM 不等式找出 b_1 ? 请仔细思考之后再看 (b).

(b) 由 (a) 得出的递推公式为 $b_{n+1} = \frac{1}{3} \left(2b_n + \frac{2}{b_n^2} \right)$. 用计算器算到 b_3 , 写成最简分数.

(c)

- 先用计算器尝试算出 b_4 . 若做不到, 说明计算器在哪一步失效, 然后改用 Python 的 Fraction 精确算出 b_4 , 并数出 $b_1, \dots, b_4$ 的分母各有几位.
- 参考 §3.1 中 $\sqrt{2}$ 的情形 ($a_n - \sqrt{2} < \frac{a_n^2 - 2}{a_n}$), 说明: 当有理数 x 满足 $x^3 > 2$ 时,

$$0 < x - 2^{1/3} < \frac{x^3 - 2}{x^2},$$

即右边是 x 的一个误差上界估计. 它只涉及有理数. **提示:** 先比较 $(2/x^2)^3$ 与 2.

- 对 a_4, a_5, b_3, b_4 , 先检查 $x^2 > 2$ (或 $x^3 > 2$), 再用上面的估计 ($\sqrt{2}$ 用 $\frac{x^2 - 2}{x}$) 算出分母的位数和误差上界估计, 并求使这个上界 $< 10^{-k}$ 的最大整数 k (即保证误差小于 10^{-k}). 能用计算器的先用计算器; 计算器的位数不够、或两个接近的数相减后有效数字丢失时, 改用 Python. 填表:

分母位数	误差上界估计	k
a_4		
a_5		
b_3		
b_4		

- 从 a_4 到 a_5 、从 b_3 到 b_4 , 分母的位数各变成原来的几倍? k 各增加了多少? 哪个迭代的分母花得更值?

提示: 计算器算不了时, 用 `fractions.Fraction` 做精确的有理数运算; `len(str(x.denominator))` 数分母位数; `math.log10` 求 k .

- 对 $2^{1/3}$ 用 1.6 的步骤, 得到整数部分 $m_0, m_1, \dots = 1, 3, 1, 5, 1, 1, 4, \dots$. 前几个分数 p_j/q_j 由

$$p_j = m_j p_{j-1} + p_{j-2}, \quad q_j = m_j q_{j-1} + q_{j-2}, \quad p_{-1} = 1, q_{-1} = 0, p_0 = m_0, q_0 = 1$$

给出. 求出前七个分数. $p^3 - 2q^3 > 0$ 说明 $p/q > 2^{1/3}$, < 0 说明 $p/q < 2^{1/3}$. 相邻两个分数在 $2^{1/3}$ 的两侧, 所以 $2^{1/3}$ 与其中一个分数的距离小于它们之间的距离. 用这一点检验前六个分数的误差是否小于 $1/q^2$, 并与 (c) 中 b_n 的分母比较. 由此猜测: 是否每个无理数都有无穷多个分数满足 $|x - p/q| < 1/q^2$? (请借助 AI 和 Wikipedia, 以及陶哲轩的个人主页了解 Dirichlet 定理).

本节新词 (Vocabulary)

English	中文
factoring	因式分解
bounds	上下界
second term	第二项
lowest terms	最简形式
contradict	与.....矛盾

回望 (Looking back)

平方比较和残差给出有理近似的误差界; 奇偶性证明不存在平方等于 2 的有理数. 下一讲定义数列极限, 精确描述近似值的趋近. 迭代的速度与实数构造分别留到第 4、14 讲.